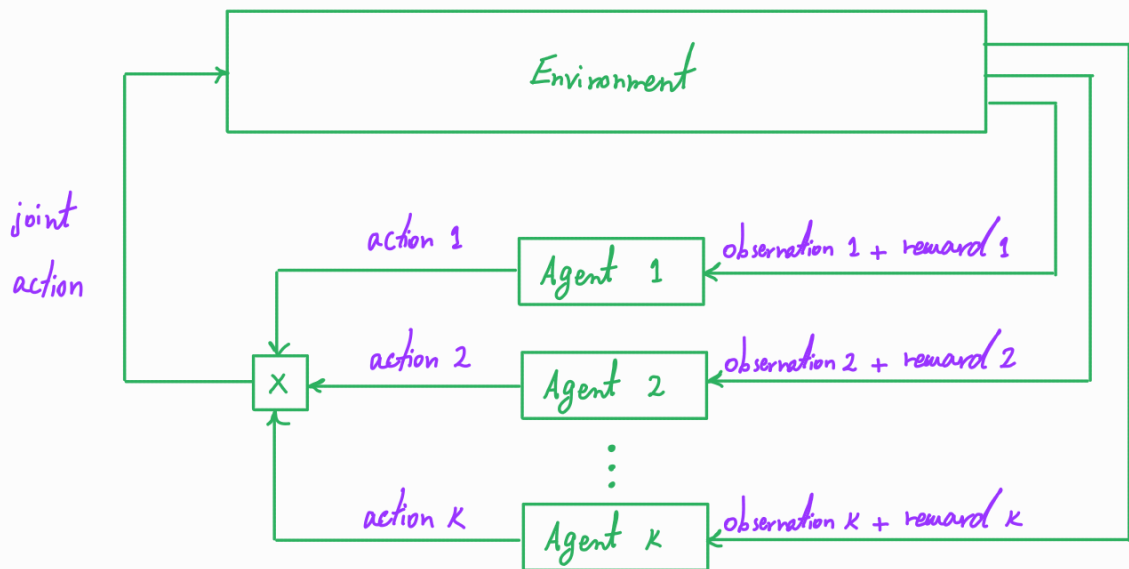
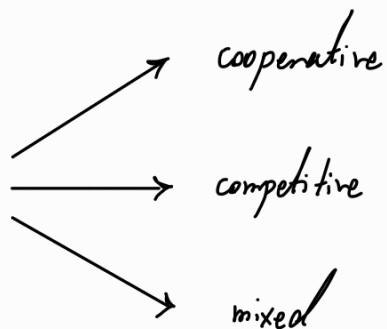


Multi-Agent Reinforcement Learning (MARL)

In MARL, there are multiple decision-makers that learn from interaction with a shared environment.



MARL settings can be generally categorized according to the relation among the agents' objectives:



* Given the multi-agent interaction, we need to use game models.

Markov Game: A Markov game (stochastic game) can be viewed as an extension of MDPs that are single-agent models to the multi-agent setting.

$$\mathcal{M} = (\mathcal{I}, \mathcal{S}, \mathcal{M}_0, \mathcal{A}_i, P, R_i)$$

$\mathcal{I}$: set of agents $\mathcal{I} = \{1, 2, \dots, K\}$

$\mathcal{S}$: state space

$\mathcal{M}_0$: initial state distribution; $\mathcal{M}_0 \in \Delta(\mathcal{S})$

$\mathcal{A}_i$: action space for agent i ; joint action space $\mathcal{A} = \prod_{i \in \mathcal{I}} \mathcal{A}_i$

P : transition function; $P: \mathcal{S} \times \mathcal{A} \rightarrow \Delta(\mathcal{S})$

R_i : reward function for agent i ; $R_i: \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$

Value Function: Each agent has a value function defined with respect to its own reward function and the joint policy.

- Let $\pi = (\pi_1, \pi_2, \dots, \pi_K)$ be the joint policy. For the infinite-horizon discounted setting,

$$V_i(\pi) = \mathbb{E}_{\substack{S_0 \sim \mathcal{M}_0 \\ A_t \sim \pi(\mathcal{Z}_t) \\ S_{t+1} \sim P(\cdot | S_t, A_t)}} \left[\sum_{t=0}^{\infty} \gamma^t R_i(S_t, A_t) \right] \quad \forall i \in \mathcal{I}$$

Goal: The learning agents' objective is to find a policy π that maximizes their value function. However, the value function couples the agents' policies.

→ Given this coupling, different solution concepts, e.g., different forms of equilibria are proposed.

Nash Equilibrium: Nash equilibrium is a state of the game where no agent can improve its gain by unilaterally deviating from its current policy. Formally, a joint policy $\pi = (\pi_1, \pi_2, \dots, \pi_K)$ is a Nash equilibrium if

$$V_i(\pi'_i, \pi_{-i}) \leq V_i(\pi) \quad \forall i \in I, \forall \pi'_i \in \Pi_i.$$

$\downarrow$
 $\pi_1, \pi_2, \dots, \pi_{i-1}, \pi_{i+1}, \dots, \pi_K$

Case Study : Alpha Go / Alpha Zero

Alpha Go is an AI Go player capable of playing the game of Go, a highly challenging board game proficiently.



Two-player Markov game: The setting of such games can be modeled by a two-player Markov game

$$\mathcal{M} = (\mathcal{S}, \mathcal{M}_0, \mathcal{A}, P, R, T),$$

where

- $\mathcal{M}_0$ is concentrated at s_0
- P is deterministic and known
- Two players with policy π_1 and policy π_2 take actions alternatively
$$\begin{array}{ccccccc} \pi_1(s_0) & \pi_2(s_1) & \pi_1(s_2) & & & & \\ \downarrow & \downarrow & \downarrow & & & & \\ s_0 & a_0 & s_1 & a_1 & s_2 & a_2 & \dots, s_T \end{array}$$

- R is sparse and known

$$R(s_T) = \begin{cases} 1 & \text{if } \pi_1 \text{ wins} \\ -1 & \text{if } \pi_2 \text{ wins} \end{cases}$$

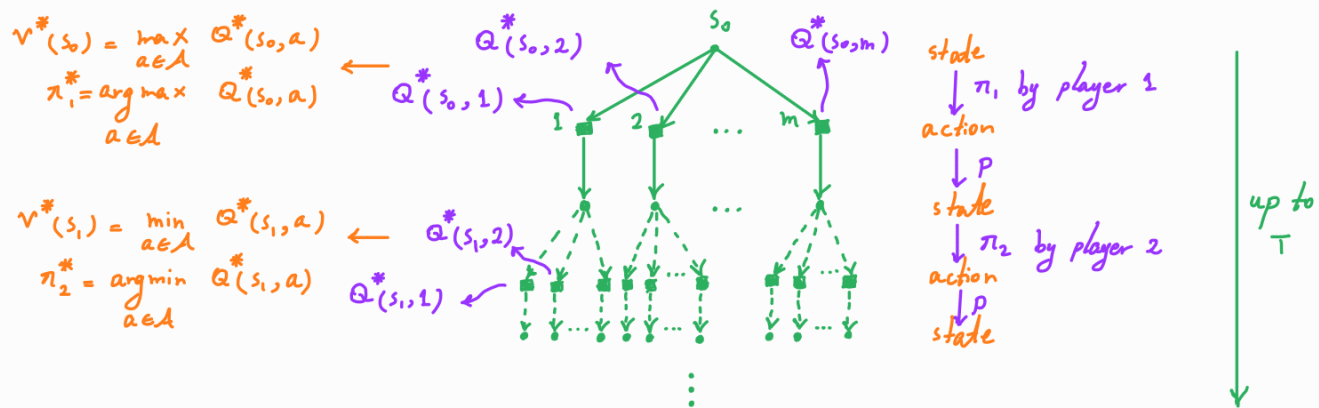
The objective is formulated as a min-max optimization problem:

$$\max_{\pi_1} \min_{\pi_2} \mathbb{E}_{\pi_1, \pi_2} [R(s_T)]$$

$\downarrow$
 $\mathcal{M} \ni P$ deterministic

zero-sum game

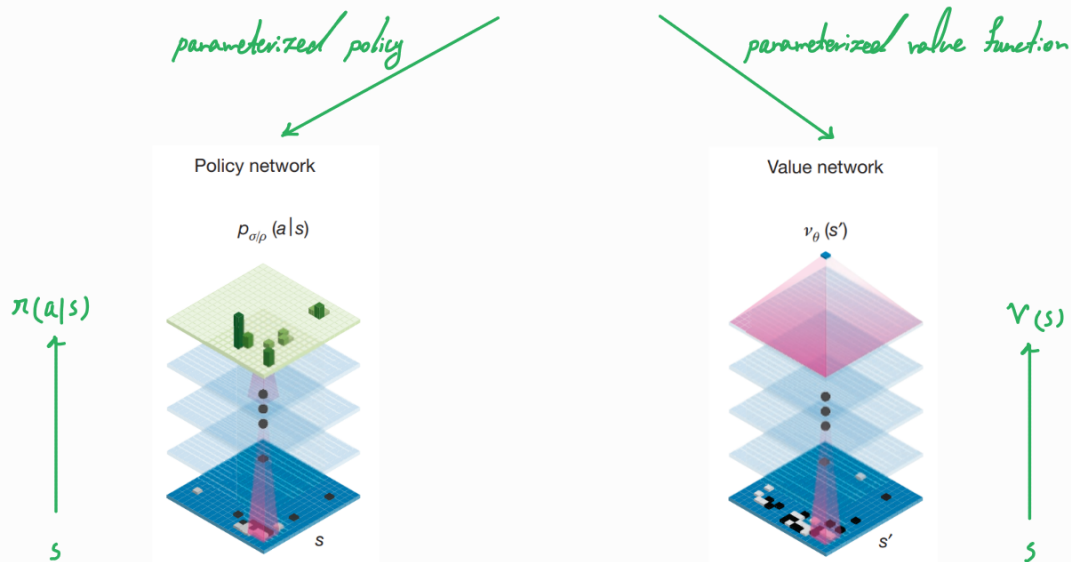
Note: Given that the model (the transition function and the reward function) are known, one can apply dynamic programming to solve the min-max optimization, i.e., compute optimal policies.



However, this is computationally prohibitive since in the game of Go,

$$T \approx 150, |A| \approx 250, |S| \approx |A|^T.$$

→ Learning + function approximation



Imitation Learning: In the first training phase of Alpha Go, a large data set of expert demonstrations are used to train an initial policy through imitation learning.

- The data set consists of

$$D = \{(s_i, a_i)\}_{i=1}^N \quad \text{where } N = 30 \text{ million}$$

- The policy is learned using MLE objective

behavior cloning

$$\bar{\pi} = \arg \min_{\pi \in \Pi} \sum_{i=1}^N -\log(\pi(a_i | s_i))$$

and stochastic gradient descent

$$\theta_{t+1} = \theta_t - \eta \frac{1}{|B|} \sum_{(s,a) \in B} -\nabla_{\theta} (\log \pi_{\theta}(a|s))$$

- * This initial policy attained 57% prediction accuracy on the test data and 11% win rate against an open-source Go program.

Policy Gradient: In the second training phase of Alpha Go, the initial policy learned from imitation learning is improved through policy gradient.

- Initialize policy $\pi_0 = \bar{\pi}$ ← from behavior cloning

- For $i = 0, 1, 2, \dots$

- Randomly select a previous policy π_{θ_j} , $j < i$

- Let the agent play $\pi_{\theta_i}^{\rightarrow \pi_1}$ against $\pi_{\theta_j}^{\rightarrow \pi_2}$ and observe the trajectory

Fictitious play ←

$(s_0, a_0, s_1, a_1, s_2, a_2, \dots, s_T)$
↑ ↑ ↑
 π_1 π_2 π_1

- Update θ

$$\theta_{i+1} = \theta_i + \eta \sum_{\substack{t=0,2,4,\dots \\ \text{time indices played by } \pi_{\theta_i}}} \nabla_{\theta} \log \pi_{\theta_i}(a_t | s_t) \overbrace{R(s_T)}^{\text{return} = R(s_T)}$$

* This second policy attained 85% win rate against an open-source Go program.

Value Learning: In the third training phase of AlphaGo, the value of the second policy $\bar{\pi}$ is approximated through supervised learning by playing the same policy against itself.

• The data set consists of

$$D' = \{(s_i, y_i)\}_{i=1}^{N'} \quad \text{where } N' = 30 \text{ million}$$

• The value is learned using least square regression

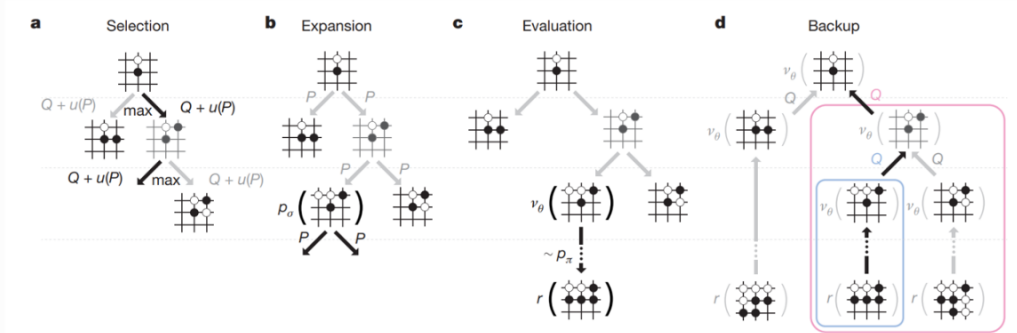
$$\hat{V}^{\tilde{\pi}} = \min_{\phi} \sum_{(s_i, y_i) \in D'} (V_{\phi}(s_i) - y_i)^2$$

and stochastic gradient descent

$$\phi_{t+1} = \phi_t - \eta' \frac{1}{|B'|} \sum_{(s, y) \in B'} (V_{\phi}(s) - y) \nabla_{\phi} V_{\phi}(s)$$

Monte-Carlo Tree search: The training is combined with an efficient and adaptive tree search algorithm.

expand promising or under-explored nodes



* This final policy attained 99.8% win rate against other Go programs and defeated a human champion.